Preflight request: ❷

```
OPTIONS /api/posts HTTP/1.1
User-Agent: Chrome
Host: 127.0.0.1:9999
Accept: */*
Origin: http://localhost:1111
Access-Control-Request-Method: GET
Access-Control-Request-Headers: Timezone-Offset, Sample-Source
```

Preflight response: ❸

```
HTTP/1.1 204 No Content
Access-Control-Allow-Origin: http://localhost:1111
Access-Control-Allow-Methods: GET, DELETE
Access-Control-Allow-Headers: Timezone-Offset, Sample-Source
```

Actual request: ❹

```
GET /api/posts HTTP/1.1
User-Agent: Chrome
Host: 127.0.0.1:9999
Accept: */*
Origin: http://localhost:1111
Timezone-Offset: 300
Sample-Source: Cors in Action
```

> **NOTE** If the requested header is a simple header, it's not required to be included in the Access-Control-Allow-Headers response header. But I recommend including simple headers to avoid confusion.

Listing 4.8 modifies the sample code to respond to the Access-Control-Request-Headers header. The code sets the Access-Control-Allow-Headers header to the value of the headers you support. It also adds the `GET` method to the list of allowed HTTP methods, because the custom headers are sent on a `GET` request. The sample will work fine even without adding `GET` to the Access-Control-Allow-Methods header (because `GET` is a simple method), but I like to include it to avoid confusion. Restart the server and reload the page, and the blog posts should reappear.

Listing 4.8 Responding to Access-Control-Request-Headers

```
var handleCors = function(req, res, next) {
  res.set('Access-Control-Allow-Origin', 'http://localhost:1111');
  if (isPreflight(req)) {
    res.set('Access-Control-Allow-Methods', 'GET, DELETE');
    res.set('Access-Control-Allow-Headers',
            'Timezone-Offset, Sample-Source');
    res.status(204).end();
    return;
  }
  next();
};
```

Adds GET to the Access-Control-Allow-Methods response header

Adds Access-Control-Allow-Headers to preflight response

After the server responds to the preflight request, the browser inspects the preflight response and verifies that the server is granting the appropriate permissions. If the preflight response checks out, the browser sends the actual request to the server.